

Difusão viscosa 2D: particionamento e balanceamento de carga

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

“Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade. Desconsidere a pressão e quaisquer forças externas. Utilize diferenças finitas para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das cláusulas `schedule` e `collapse` no desempenho da execução paralela.”

2 Fundamentação teórica

2.1 Difusão viscosa e diferenças finitas

Considerando apenas a viscosidade, foi adotada a equação de difusão para uma componente escalar da velocidade:

$$\frac{\partial u}{\partial t} = \nu \nabla^2 u,$$

em que u representa a componente de velocidade e ν é a viscosidade cinemática. Nessa simplificação, foram omitidas a advecção, a pressão e as forças externas. Portanto, o programa não resolve o sistema completo de Navier–Stokes incompressível.

A discretização utiliza Euler explícito no tempo e diferenças centrais no espaço, formando o esquema FTCS. Para uma malha quadrada de espaçamento h :

$$u_{i,j}^{n+1} = u_{i,j}^n + r \left(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n \right), \quad r = \frac{\nu \Delta t}{h^2}.$$

Esse stencil de cinco pontos calcula o novo valor a partir da célula central e de seus quatro vizinhos no instante anterior. Foram utilizados $\nu = 0,1$, $\Delta t = 0,2$ e $h = 1$, resultando em $r = 0,02$. A condição de estabilidade em duas dimensões é $0 \leq r \leq 1/4$, atendida pelos parâmetros utilizados.

Em um campo constante, o laplaciano discreto é zero e o valor permanece inalterado, desde que as bordas tenham o mesmo valor. Em uma perturbação localizada, a difusão reduz o pico e espalha o campo para as células vizinhas.

2.2 Particionamento e políticas de escalonamento

A diretiva `parallel` cria a equipe de threads, enquanto `omp for` distribui as iterações entre elas. A cláusula `schedule` define essa distribuição:

- `static`: divide as iterações em blocos aproximadamente iguais. Sem chunk explícito, cada thread recebe no máximo um bloco. Apresenta baixo custo de gerenciamento e é adequado para trabalho uniforme.
- `dynamic,k`: distribui blocos de k iterações conforme as threads terminam os anteriores. Pode reduzir o desbalanceamento, mas exige gerenciamento a cada bloco.
- `guided,k`: também entrega novos blocos conforme a conclusão do trabalho, porém começa com blocos maiores e reduz seus tamanhos. O valor k é o mínimo, exceto no último bloco.

O chunk conta iterações do laço distribuído. Blocos pequenos aumentam as oportunidades de balanceamento e o custo de distribuição; blocos grandes reduzem esse custo, mas deixam menos trabalho disponível para redistribuir. Nesta tarefa, todas as células internas executam a mesma operação.

2.3 Efeito de `collapse(2)`

Sem `collapse`, apenas o laço externo é distribuído: cada iteração corresponde a uma linha. Com `collapse(2)`, os laços de i e j formam um único espaço de iterações, no qual cada unidade corresponde a uma célula.

Para $N = 512$, são 510 linhas ou $510^2 = 260.100$ células internas. Assim, chunk 16 representa 16 linhas sem `collapse` e 16 células com a cláusula. Essa diferença precisa ser considerada na comparação de desempenho. O laço temporal permanece sequencial, pois cada passo depende do campo produzido pelo anterior.

3 Implementação

3.1 Organização das versões

Foram implementadas sete versões independentes em C11: uma referência sequencial e seis variantes OpenMP. Cada arquivo contém a inicialização, a atualização do campo, a medição de tempo e a exportação. A separação facilita a comparação das alterações nos pragmas.

A malha tem 512×512 pontos e são realizados 500 passos. Esses valores são definidos no código por `tamanho_grid` e `num_passos_tempo`. A interface recebe `[modo [arquivo-campo]]`: modo 0 para a perturbação gaussiana, modo 1 para o campo nulo e modo 2 para o campo unitário. O modo padrão é 0. São verificados os argumentos, as alocações e a escrita do arquivo.

A gaussiana tem amplitude 0,1, centro na malha e $\sigma = N/10$. As bordas são fixas: zero nos modos 0 e 1, e um no modo 2. Dois vetores de `double`, acessados por `c = i*N+j`, armazenam os campos. As bordas são inicializadas nos dois vetores e ficam fora dos laços de atualização.

Versão	Política	Unidade distribuída
v0	Sequencial	Sem distribuição
v1	static	Linha
v2	static, collapse(2)	Célula
v3	dynamic, 16	Bloco de 16 linhas
v4	dynamic, 16, collapse(2)	Bloco de 16 células
v5	guided, 16, collapse(2)	Bloco decrescente de células
v6	guided, 16	Bloco decrescente de linhas

Table 1: Versões completas e independentes. Em guided, 16 é o tamanho mínimo, salvo o último bloco.

3.2 Atualização do campo e sincronização

O vetor apontado por *u* contém o campo atual; o vetor apontado por *v* recebe o próximo campo. Eles representam dois instantes da mesma componente de velocidade. Em cada passo, as threads leem apenas *u* e escrevem em posições distintas de *v*. Atualizar o próprio *u* misturaria valores de instantes diferentes e introduziria dependências entre as iterações.

Uma única região `parallel` envolve os 500 passos. Dentro dela, cada thread percorre o laço temporal com seu contador local, e `omp for` divide as atualizações espaciais. O trecho abaixo mostra a versão v1; as demais variantes paralelas alteram a política e a presença de `collapse(2)`.

```

68     for (int t = 0; t < num_passos_tempo; ++t) {
69         #pragma omp for schedule(static)
70         for (int i = 1; i < tamanho_grid-1; ++i) {
71             for (int j = 1; j < tamanho_grid-1; ++j) {
72                 size_t c = (size_t)i*tamanho_grid+j;
73                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
74                     +tamanho_grid]-4*u[c]);
75             }
76         } /* Barreira: todas as escritas terminam antes da troca. */
77         #pragma omp single
78         {
79             double *tmp = u; u = v; v = tmp;
80         } /* Barreira: todas as threads veem a troca antes do próximo
           passo. */
81     }

```

Listing 1: Laço temporal e troca de buffers na versão v1.

A barreira implícita ao final de `omp for` garante que todas as células foram atualizadas antes da troca. A diretiva `single` permite que uma thread troque os ponteiros, sem copiar a matriz. Sua barreira final garante que o próximo passo comece com os ponteiros já trocados. Portanto, retirar qualquer uma dessas barreiras com `nowait`, mantendo o restante do código, comprometeria a correção. Não é necessário usar `critical` ou `atomic` por célula, pois cada posição de saída tem um único escritor.

Foi utilizado `default(none)` para explicitar o compartilhamento das variáveis sem escopo predefinido. Os ponteiros `u` e `v`, os parâmetros e o registro de threads são compartilhados. Os contadores locais e as variáveis `c` e `tmp` são privados. O número efetivo de threads é obtido com `omp_get_num_threads()` dentro de um `single` inicial.

3.3 Compilação e execução

A compilação utiliza GCC com otimização `-O2`, avisos habilitados e `-fopenmp` apenas nas variantes paralelas. Os exemplos abaixo são executados no diretório `teste11/`:

```
1 gcc -std=c11 -O2 -Wall -Wextra v0_seq.c -lm -o ns_v0
2 gcc -std=c11 -O2 -Wall -Wextra -fopenmp v1_static.c -lm -o ns_v1
3 bash test.sh
4 ./ns_v0 1
5 ./ns_v0 2
6 OMP_NUM_THREADS=4 ./ns_v1 0 campo.txt
```

Listing 2: Compilação, validação e execução.

Nas versões `dynamic` e `guided`, a macro `CHUNK_SIZE` vale 16 por padrão e pode ser alterada na compilação com `-DCHUNK_SIZE=valor`. Essa opção foi utilizada no estudo de chunks equivalentes.

4 Resultados

4.1 Validação numérica

O script `test.sh` compila as sete versões com `-std=c11 -O2 -Wall -Wextra -Werror`, acrescentando `-fopenmp` às paralelas e `-lm` na ligação. Foram aprovadas 30 comparações exatas dos campos completos de 512×512 pontos: para cada variante paralela, o modo gaussiano foi testado com 1, 2 e 4 threads; os modos nulo e unitário, com quatro threads.

O campo parado permaneceu nulo e o campo constante permaneceu unitário. As versões paralelas produziram os mesmos campos da referência sequencial. Também foram verificadas a não negatividade do campo perturbado, seu pico abaixo de 0,1 e a rejeição de argumentos inválidos e de um caminho de saída inexistente.

A Figura 1 compara a gaussiana inicial com o campo após 500 passos. A redução do pico e o aumento do raio quadrático médio indicam o espalhamento da perturbação. Esses testes verificam os casos executados; a estabilidade do esquema decorre também da condição sobre r apresentada na fundamentação.

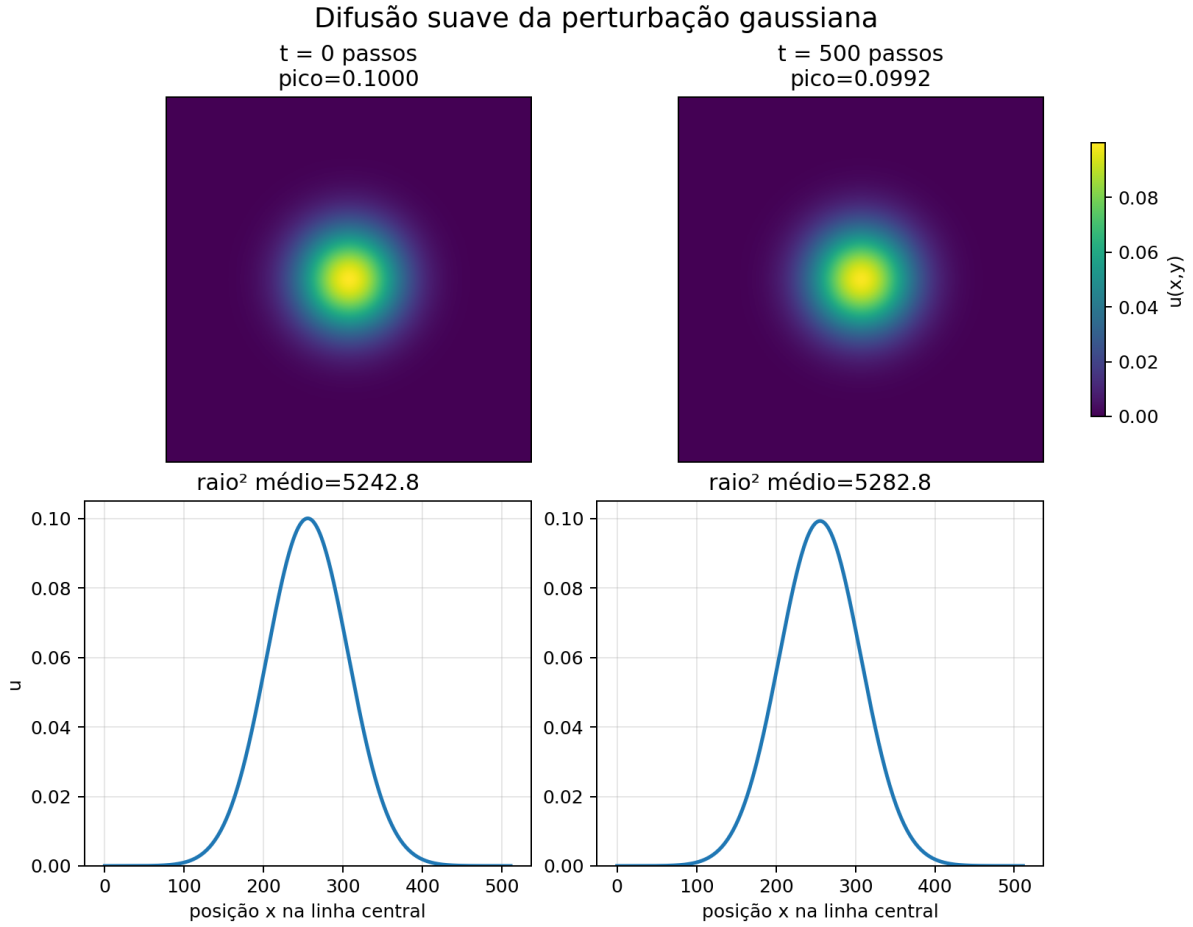


Figure 1: Difusão da perturbação gaussiana no início e após 500 passos.

4.2 Medição de desempenho

O benchmark foi executado com GCC 13.3.0 no Intel Core i7-14700HX sob WSL2, usando uma grade 512×512 , 500 passos e uma varredura de 1 a 28 threads. Esse é o número de processadores lógicos disponíveis no ambiente virtualizado. Cada ponto de 1 a 25 threads usa a mediana de cinco repetições após aquecimento. Como a primeira varredura apresentou picos abruptos no final, os pontos de 26 a 28 threads foram repetidos dez vezes sob as mesmas condições e substituídos por essas medianas confirmadas. Foram usadas as opções `-std=c11`, `-O2`, `-Wall` e `-Wextra`, além de `-lm`; nas variantes paralelas, foi acrescentado `-fopenmp`. Foram fixados `OMP_DYNAMIC=FALSE`, `OMP_PROC_BIND=close` e `OMP_PLACES=cores`.

O cronômetro `CLOCK_MONOTONIC` cobre a região paralela, o stencil, as barreiras e a troca de ponteiros. Alocação, inicialização, cálculo dos diagnósticos (mínimo, pico e soma) e exportação ficam fora da temporização. A Tabela 2 apresenta medianas selecionadas; as Figuras 2 e 3 mostram toda a varredura e o speedup $S(p) = T_{seq}/T_p$, em que T_{seq} é o tempo da versão sequencial e T_p é o tempo com p threads. O conjunto consolidado está em `runs/resultado-confirmado/`; as duas execuções de origem permanecem preservadas para rastreabilidade.

Versão	1 thread	2 threads	4 threads	8 threads	14 threads	28 threads
v1 static	0,0970	0,0531	0,0289	0,0223	0,0207	0,0185
v2 static + collapse	0,1312	0,0722	0,0411	0,0326	0,0280	0,0488
v3 dynamic	0,1064	0,0621	0,0337	0,0220	0,0228	0,0276
v4 dynamic + collapse	0,2241	0,3662	0,3482	0,3342	0,3007	0,3225
v5 guided + collapse	0,1318	0,0706	0,0395	0,0277	0,0242	0,0329
v6 guided	0,1239	0,0588	0,0327	0,0240	0,0229	0,0269

Table 2: Tempo mediano em segundos, selecionado da varredura completa de 1 a 28 threads. A referência sequencial teve mediana de 0,1049 s.

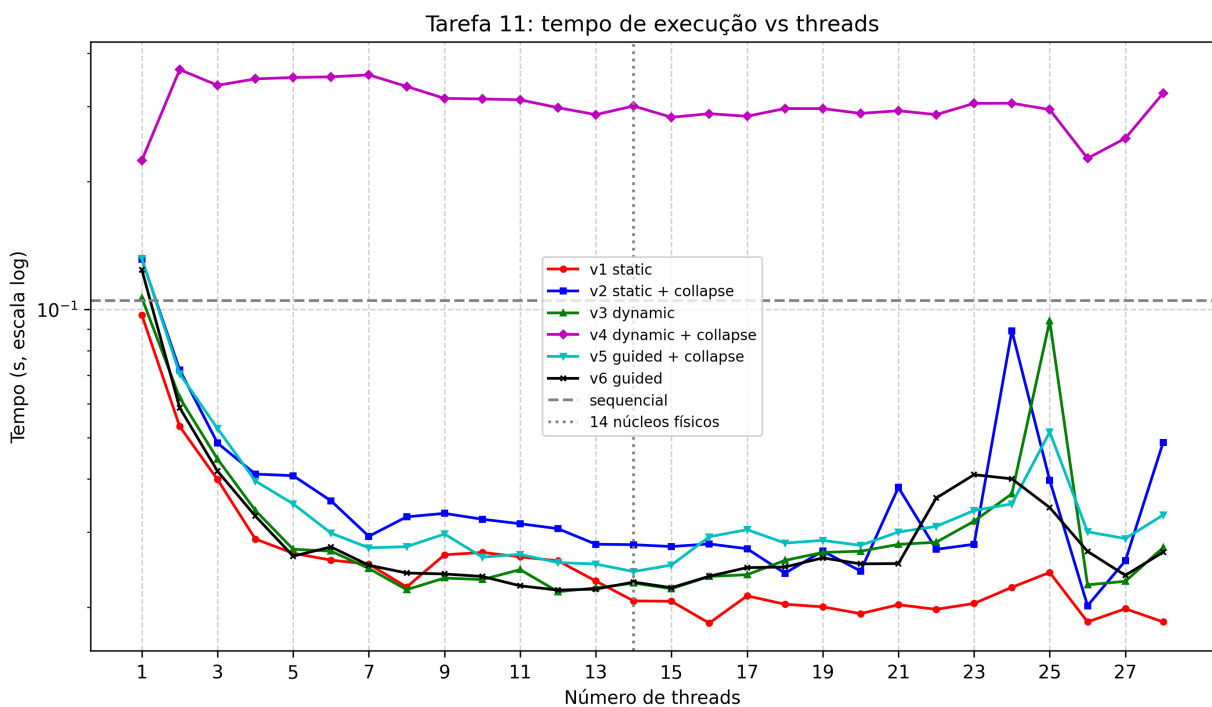


Figure 2: Tempo mediano das políticas com e sem `collapse(2)`, de 1 a 28 threads, em escala logarítmica.

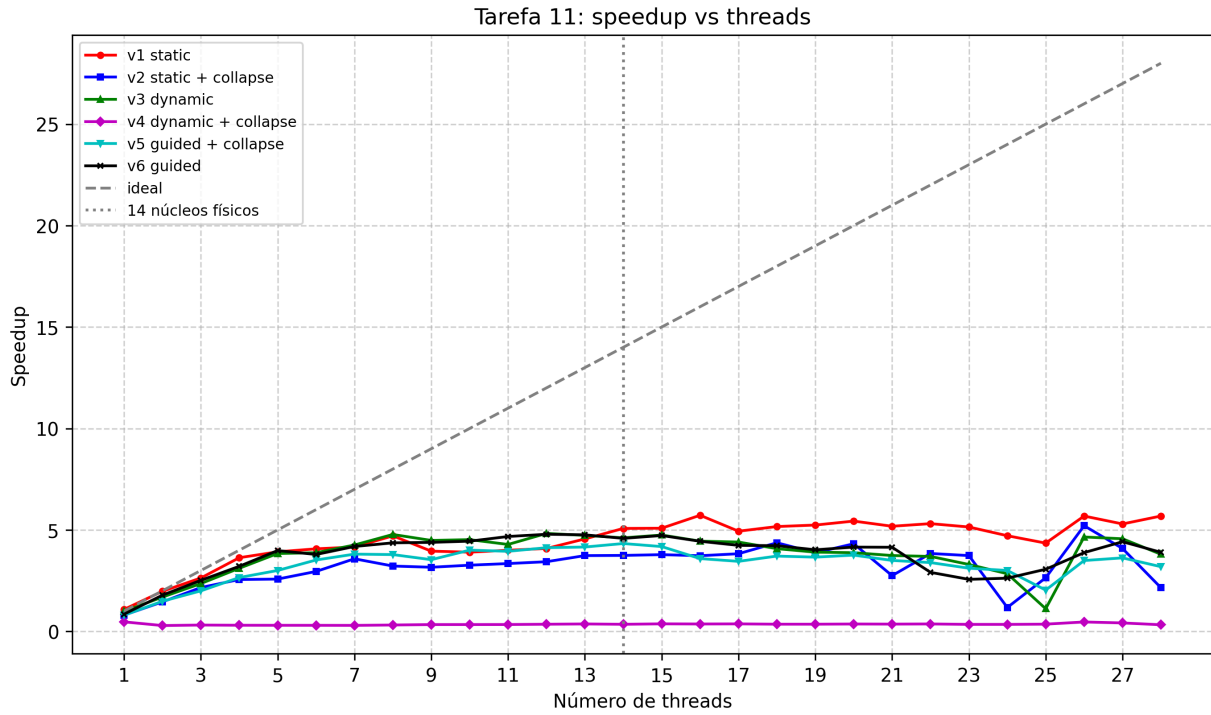


Figure 3: Speedup relativo à referência sequencial. A reta tracejada representa o caso ideal.

4.3 Variação do chunksize

Para comparar as variantes com o mesmo volume de trabalho por chunk, foi realizado um segundo experimento com oito threads. Sem collapse, os chunks foram 1, 4, 16 e 64 linhas. Com collapse, foram utilizadas 510, 2.040, 8.160 e 32.640 células, quantidades equivalentes a 1, 4, 16 e 64 linhas internas. Cada configuração teve cinco repetições; os dados estão em [runs/chunks-20260911-152008-206654/](#).

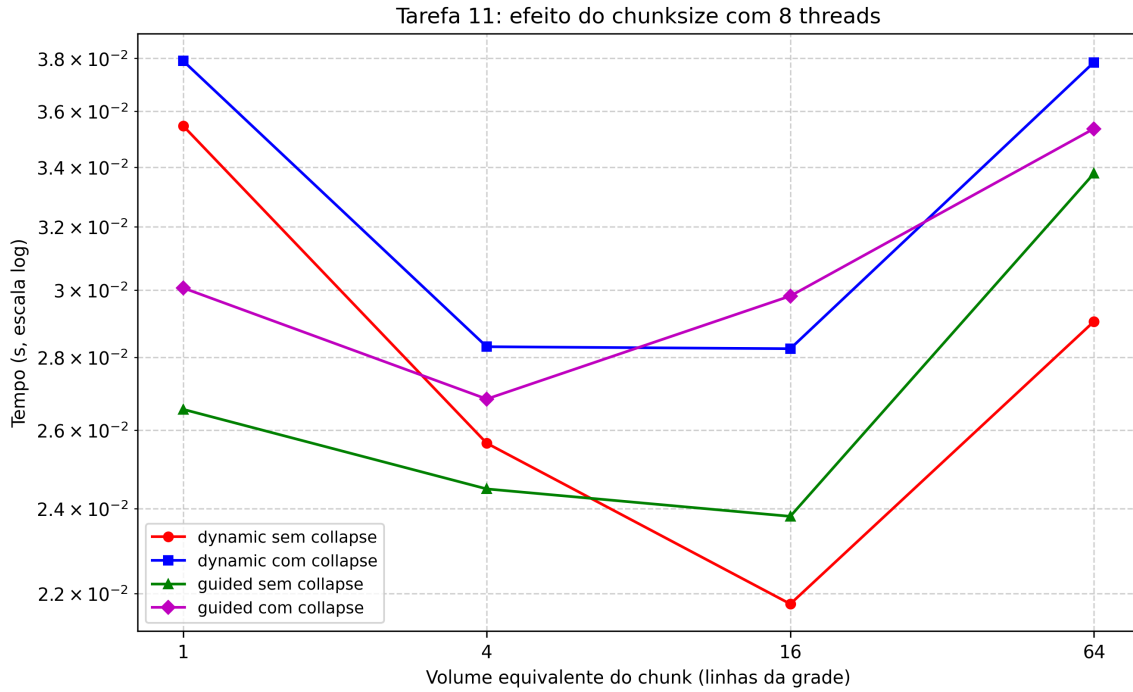


Figure 4: Efeito do chunksize com oito threads. O eixo horizontal representa o volume equivalente em linhas.

No *dynamic*, o melhor resultado sem collapse ocorreu com 16 linhas por bloco (0,0218 s); com collapse, quatro e 16 linhas equivalentes ficaram próximos (0,0283 s). No *guided*, o menor tempo sem collapse ocorreu com mínimo de 16 linhas (0,0238 s); com collapse, o melhor mínimo foi de quatro linhas equivalentes (0,0268 s). Nos dois escalonamentos, o maior chunk testado, de 64 linhas equivalentes, foi mais lento que os tamanhos intermediários.

5 Discussão

5.1 Granularidade e custo de distribuição

A comparação entre v3 e v4 mostra por que o valor do chunk precisa ser interpretado junto com collapse. Com *dynamic*, 16, o número de blocos por passo é:

$$\text{sem collapse: } \left\lceil \frac{510}{16} \right\rceil = 32, \quad \text{com collapse: } \left\lceil \frac{260.100}{16} \right\rceil = 16.257.$$

No primeiro caso, um bloco completo reúne 8.160 atualizações; no segundo, apenas 16. Ao longo de 500 passos, são 16.000 e 8.128.500 blocos, respectivamente. Eles são unidades de distribuição do laço, e não novas threads.

Com quatro threads, v3 levou 0,0337 s e v4 levou 0,3482 s. A multiplicação dos blocos aumenta o trabalho de gerenciamento para a mesma quantidade de atualizações. O experimento com oito threads reforça essa interpretação: ao passar de 16 células para 8.160 células por chunk, a versão *dynamic* com collapse passou de 0,3342 s, na varredura principal, para 0,0283 s, no estudo de

chunks. Assim, a piora de v4 está associada à granularidade adotada e não permite concluir que `collapse` seja sempre inadequado.

Em `guided`, o chunk é um mínimo e os blocos iniciais são maiores. Por isso, a contagem de blocos não segue a divisão fixa acima. Essa diferença ajuda a explicar por que v5 apresentou tempos menores que v4, embora ambas distribuam células.

5.2 Localidade e limites de escalabilidade

A versão v1, com `static` por linhas, obteve o menor tempo da varredura: 0,0183 s com 16 threads, contra 0,1049 s da referência sequencial. O speedup foi 5,72 e a eficiência, $E(p) = S(p)/p$, foi aproximadamente 35,8%. Com quatro threads, o tempo foi 0,0289 s e o speedup foi 3,63.

A carga uniforme favorece a distribuição estática, que exige pouco gerenciamento. O acesso `c=i*N+j` percorre posições contíguas ao variar j e permite reutilizar vizinhos no cache. Os menores tempos das versões `dynamic` e `guided` por linhas ocorreram com 12 threads: 0,0217 s e 0,0219 s, respectivamente.

Já existem 510 linhas internas para distribuir entre até 28 threads. Portanto, `collapse(2)` não é necessário para fornecer trabalho a toda a equipe nesta malha. Ele altera a granularidade e pode afetar os índices gerados e a vetorização. Esses efeitos individuais não foram medidos.

Os dois buffers ocupam 4 MiB. Em uma linha de cache de 64 bytes cabem oito valores `double`; escritas de threads distintas na mesma linha podem provocar falso compartilhamento. Entretanto, os tempos, isoladamente, não identificam esse efeito nem comprovam saturação da memória RAM. Parte dos dados pode ser reutilizada em cache.

Entre 16 e 28 threads, v1 apresentou pouca diferença nas medianas: 0,0183 s e 0,0185 s. O trabalho por thread diminui, mas as barreiras e o gerenciamento continuam presentes. Os picos da primeira varredura em 26–28 threads não se repetiram de forma sistemática na confirmação. A virtualização e o compartilhamento dos recursos de hardware também podem afetar os tempos; sua contribuição não foi isolada. Assim, 16 threads corresponde ao menor tempo medido, sem estabelecer uma quantidade ótima para outras malhas ou máquinas.

6 Conclusão

A implementação preservou os campos nulo e unitário e apresentou o espalhamento da perturbação gaussiana. Nos casos testados, as seis versões paralelas produziram campos idênticos ao da referência sequencial. A separação dos buffers e as barreiras entre atualização e troca de ponteiros mantêm a dependência entre os passos temporais.

Para a malha e o ambiente avaliados, `schedule(static)` sem `collapse` apresentou o menor tempo, com speedup de 5,72 em 16 threads. A versão `dynamic`, 16 com `collapse` teve os maiores tempos porque distribuiu blocos de apenas 16 células. O estudo com chunks equivalentes mostrou que aumentar esse volume reduz substancialmente o custo observado. Portanto, a escolha do escalonamento deve considerar o custo das iterações e a unidade de trabalho distribuída.

A Código-fonte

O apêndice reúne os sete códigos completos utilizados nos testes e nas medições.

```
1 // gcc -std=c11 -O2 -Wall -Wextra v0_seq.c -lm -o ns_v0
2 #ifdef _OPENMP
3 #error "Compile a referencia sequencial sem -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #include <errno.h>
7 #include <math.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12
13 static int integer(const char *text, int minimum, int maximum, int *out) {
14     char *end;
15     errno = 0;
16     long value = strtol(text, &end, 10);
17     if (errno || end == text || *end || value < minimum || value > maximum
18         )
19         return 0;
20     *out = (int)value;
21     return 1;
22 }
23
24 static double agora(void) {
25     struct timespec t;
26     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
27         perror("clock_gettime");
28         exit(EXIT_FAILURE);
29     }
30     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
31 }
32
33 int main(int argc, char **argv) {
34     const int tamanho_grid = 512;
35     const int num_passos_tempo = 500;
36     int mode = 0, threads = 1;
37     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
38         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
39         return EXIT_FAILURE;
40     }
41     size_t count = (size_t)tamanho_grid * tamanho_grid;
42     double *u = malloc(count * sizeof(*u));
43     double *v = malloc(count * sizeof(*v));
44     if (!u || !v) {
45         fprintf(stderr, "Falha de alocacao\n");
46     }
```

```

45     free(u); free(v);
46     return EXIT_FAILURE;
47 }
48 const double nu = 0.1, dt = 0.2, dx = 1.0;
49 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
50 for (int i = 0; i < tamanho_grid; ++i) {
51     for (int j = 0; j < tamanho_grid; ++j) {
52         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
53             - 1) / 2.0;
54         double sigma = tamanho_grid / 10.0;
55         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
56             tamanho_grid-1;
57         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
58             (mode == 1 || boundary ? 0.0 :
59                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
60     }
61 }
62 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
63 memcpy(v, u, count * sizeof(*v));
64 double start = agora();
65 {
66     for (int t = 0; t < num_passos_tempo; ++t) {
67         for (int i = 1; i < tamanho_grid-1; ++i) {
68             for (int j = 1; j < tamanho_grid-1; ++j) {
69                 size_t c = (size_t)i*tamanho_grid+j;
70                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
71                     +tamanho_grid]-4*u[c]);
72             }
73         } /* Barreira: todas as escritas terminam antes da troca. */
74         {
75             double *tmp = u; u = v; v = tmp;
76         } /* Barreira: todas as threads veem a troca antes do próximo
77             passo. */
78     }
79 }
80 double elapsed = agora() - start;
81 double peak = u[0], minimum = u[0], sum = 0.0;
82 for (size_t c = 0; c < count; ++c) {
83     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
84         ];
85 }
86 printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
87     =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
88     "v0_seq", tamanho_grid, num_passos_tempo, mode, threads, r,
89     elapsed, minimum, peak, sum);
90 int status = EXIT_SUCCESS;
91 if (argc == 3) {
92     FILE *file = fopen(argv[2], "w");
93     if (!file) { perror(argv[2]); status = EXIT_FAILURE; }

```

```

87     else {
88         for (size_t c = 0; c < count; ++c)
89             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
90                 ' : '\n') < 0)
91                 status = EXIT_FAILURE;
92             if (fclose(file)) status = EXIT_FAILURE;
93         }
94     }
95     free(u); free(v);
96     return status;
97 }

```

Listing 3: v0_seq.c: referência sequencial

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp vi_static.c -lm -o ns_vi
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #include <errno.h>
7 #include <math.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12 #include <omp.h>
13
14 static int integer(const char *text, int minimum, int maximum, int *out) {
15     char *end;
16     errno = 0;
17     long value = strtol(text, &end, 10);
18     if (errno || end == text || *end || value < minimum || value > maximum
19         )
20         return 0;
21     *out = (int)value;
22     return 1;
23 }
24
25 static double agora(void) {
26     struct timespec t;
27     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
28         perror("clock_gettime");
29         exit(EXIT_FAILURE);
30     }
31     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
32 }
33
34 int main(int argc, char **argv) {
35     const int tamanho_grid = 512;
36     const int num_passos_tempo = 500;
37     int mode = 0, threads = 1;
38     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
39         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
40         return EXIT_FAILURE;
41     }
42     size_t count = (size_t)tamanho_grid * tamanho_grid;
43     double *u = malloc(count * sizeof(*u));
44     double *v = malloc(count * sizeof(*v));
45     if (!u || !v) {
46         fprintf(stderr, "Falha de alocao\n");
47         free(u); free(v);
48         return EXIT_FAILURE;
49     }

```

```

48 }
49 const double nu = 0.1, dt = 0.2, dx = 1.0;
50 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
51 for (int i = 0; i < tamanho_grid; ++i) {
52     for (int j = 0; j < tamanho_grid; ++j) {
53         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
54             - 1) / 2.0;
55         double sigma = tamanho_grid / 10.0;
56         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
57             tamanho_grid-1;
58         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
59             (mode == 1 || boundary ? 0.0 :
60                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
61     }
62 }
63 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
64 memcpy(v, u, count * sizeof(*v));
65 double start = agora();
66 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
67     num_passos_tempo,r,threads)
68 {
69     #pragma omp single
70     threads = omp_get_num_threads();
71     for (int t = 0; t < num_passos_tempo; ++t) {
72         #pragma omp for schedule(static)
73         for (int i = 1; i < tamanho_grid-1; ++i) {
74             for (int j = 1; j < tamanho_grid-1; ++j) {
75                 size_t c = (size_t)i*tamanho_grid+j;
76                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
77                     +tamanho_grid]-4*u[c]);
78             }
79         } /* Barreira: todas as escritas terminam antes da troca. */
80         #pragma omp single
81         {
82             double *tmp = u; u = v; v = tmp;
83         } /* Barreira: todas as threads veem a troca antes do próximo
84             passo. */
85     }
86 }
87 double elapsed = agora() - start;
88 double peak = u[0], minimum = u[0], sum = 0.0;
89 for (size_t c = 0; c < count; ++c) {
90     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c]
91 ];
92 }
93 printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
94     =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
95     "v1_static", tamanho_grid, num_passos_tempo, mode, threads, r,
96     elapsed, minimum, peak, sum);

```

```

89     int status = EXIT_SUCCESS;
90     if (argc == 3) {
91         FILE *file = fopen(argv[2], "w");
92         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
93         else {
94             for (size_t c = 0; c < count; ++c)
95                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
96                     ' : '\n') < 0)
97                     status = EXIT_FAILURE;
98             if (fclose(file)) status = EXIT_FAILURE;
99         }
100     }
101     free(u); free(v);
102     return status;
103 }

```

Listing 4: v1_static.c: static por linhas

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v2_static_collapse.c -lm -o
   ns_v2
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #include <errno.h>
7 #include <math.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12 #include <omp.h>
13
14 static int integer(const char *text, int minimum, int maximum, int *out) {
15     char *end;
16     errno = 0;
17     long value = strtol(text, &end, 10);
18     if (errno || end == text || *end || value < minimum || value > maximum
19         )
20         return 0;
21     *out = (int)value;
22     return 1;
23 }
24
25 static double agora(void) {
26     struct timespec t;
27     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
28         perror("clock_gettime");
29         exit(EXIT_FAILURE);
30     }
31     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
32 }
33
34 int main(int argc, char **argv) {
35     const int tamanho_grid = 512;
36     const int num_passos_tempo = 500;
37     int mode = 0, threads = 1;
38     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
39         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
40         return EXIT_FAILURE;
41     }
42     size_t count = (size_t)tamanho_grid * tamanho_grid;
43     double *u = malloc(count * sizeof(*u));
44     double *v = malloc(count * sizeof(*v));
45     if (!u || !v) {
46         fprintf(stderr, "Falha de alocao\n");
47         free(u); free(v);

```

```

47     return EXIT_FAILURE;
48 }
49 const double nu = 0.1, dt = 0.2, dx = 1.0;
50 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
51 for (int i = 0; i < tamanho_grid; ++i) {
52     for (int j = 0; j < tamanho_grid; ++j) {
53         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
54             - 1) / 2.0;
55         double sigma = tamanho_grid / 10.0;
56         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
57             tamanho_grid-1;
58         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
59             (mode == 1 || boundary ? 0.0 :
60                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
61     }
62 }
63 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
64 memcpy(v, u, count * sizeof(*v));
65 double start = agora();
66 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
67     num_passos_tempo,r,threads)
68 {
69     #pragma omp single
70     threads = omp_get_num_threads();
71     for (int t = 0; t < num_passos_tempo; ++t) {
72         #pragma omp for collapse(2) schedule(static)
73         for (int i = 1; i < tamanho_grid-1; ++i) {
74             for (int j = 1; j < tamanho_grid-1; ++j) {
75                 size_t c = (size_t)i*tamanho_grid+j;
76                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
77                     +tamanho_grid]-4*u[c]);
78             }
79         } /* Barreira: todas as escritas terminam antes da troca. */
80         #pragma omp single
81         {
82             double *tmp = u; u = v; v = tmp;
83         } /* Barreira: todas as threads veem a troca antes do próximo
84             passo. */
85     }
86 }
87 double elapsed = agora() - start;
88 double peak = u[0], minimum = u[0], sum = 0.0;
89 for (size_t c = 0; c < count; ++c) {
90     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c]
91 ];
92 }
93 printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
94     =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",

```

```

88         "v2_static_collapse", tamanho_grid, num_passos_tempo, mode,
           threads, r, elapsed, minimum, peak, sum);
89     int status = EXIT_SUCCESS;
90     if (argc == 3) {
91         FILE *file = fopen(argv[2], "w");
92         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
93         else {
94             for (size_t c = 0; c < count; ++c)
95                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
', : '\n') < 0)
96                     status = EXIT_FAILURE;
97             if (fclose(file)) status = EXIT_FAILURE;
98         }
99     }
100     free(u); free(v);
101     return status;
102 }

```

Listing 5: v2_static_collapse.c: static por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v3_dynamic.c -lm -o ns_v3
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));
48     if (!u || !v) {

```

```

48     fprintf(stderr, "Falha de alocação\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for schedule(dynamic, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
91           =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
           "v3_dynamic", tamanho_grid, num_passos_tempo, mode, threads, r,
           elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
99                     ' : '\n') < 0)
100                     status = EXIT_FAILURE;
101             if (fclose(file)) status = EXIT_FAILURE;
102         }
103     }
104     free(u); free(v);
105     return status;
}

```

Listing 6: v3_dynamic.c: dynamic por linhas

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v4_dynamic_collapse.c -lm -o
   ns_v4
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));

```

```

47 if (!u || !v) {
48     fprintf(stderr, "Falha de alocação\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for collapse(2) schedule(dynamic, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c]
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
      =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
91         "v4_dynamic_collapse", tamanho_grid, num_passos_tempo, mode,
          threads, r, elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
          ' : '\n') < 0)
99                     status = EXIT_FAILURE;
100             if (fclose(file)) status = EXIT_FAILURE;
101         }
102     }
103     free(u); free(v);
104     return status;
105 }

```

Listing 7: v4_dynamic_collapse.c: dynamic por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v5_guided_collapse.c -lm -o
   ns_v5
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));

```

```

47 if (!u || !v) {
48     fprintf(stderr, "Falha de alocacao\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for collapse(2) schedule(guided, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
      =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
91         "v5_guided_collapse", tamanho_grid, num_passos_tempo, mode,
          threads, r, elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
          ' : '\n') < 0)
99                     status = EXIT_FAILURE;
100             if (fclose(file)) status = EXIT_FAILURE;
101         }
102     }
103     free(u); free(v);
104     return status;
105 }

```

Listing 8: v5_guided_collapse.c: guided por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v6_guided.c -lm -o ns_v6
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));
48     if (!u || !v) {

```

```

48     fprintf(stderr, "Falha de alocação\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for schedule(guided, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
91           =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
           "v6_guided", tamanho_grid, num_passos_tempo, mode, threads, r,
           elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
99                     ' : '\n') < 0)
100                     status = EXIT_FAILURE;
101             if (fclose(file)) status = EXIT_FAILURE;
102         }
103     }
104     free(u); free(v);
105     return status;
}

```

Listing 9: v6_guided.c: guided por linhas